TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 5: Interfacing

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 5 Topics (6 Hours | 8 Marks):

- 5.1 Communication Basics — Serial vs Parallel, Synchronous vs Asynchronous
- 5.2 Microprocessor Interfacing: I/O Addressing (Memory-Mapped vs Isolated)
- 5.3 Interrupts — Types, Vectored/Non-vectored, Interrupt-driven I/O
- 5.4 DMA (Direct Memory Access) — Operation, Modes, Block Diagram
- 5.5 Arbitration — Priority Arbitration, Daisy Chain Arbitration, Round-Robin
- 5.6 Multilevel Bus Architectures
- 5.7 Advanced Communication Protocols: I2C, SPI, UART, USB, CAN

Chapter 5: Interfacing

Q1. Define interfacing. Explain the needs of interfacing. What are the different types of interfacing?	2080 Chaitra (Regular) | 2075 Jestha (Back) [1+3+4]

ANSWER

Definition of Interfacing:

Interfacing is the process of connecting two or more different electronic or digital systems (CPU, memory, peripheral devices, sensors, actuators) so they can communicate and exchange data reliably, despite differences in signal levels, speed, data format, or protocol.

Needs of Interfacing:

- Signal Level Compatibility: Different devices operate at different logic voltage levels (e.g., 5V TTL vs 3.3V CMOS vs 1.8V low-voltage). Level shifters/translators are needed to prevent damage and ensure correct logic states.
- Data Format Conversion: CPUs use parallel data internally, but many peripherals use serial communication (UART, SPI, I2C). Interface circuits perform serial↔parallel conversion.
- Speed Matching: A fast CPU (GHz range) connected to a slow peripheral (kbps UART) needs buffering, FIFOs, and handshaking to synchronize data transfer without data loss.
- Protocol Translation: The CPU bus protocol is different from SPI, I2C, UART, USB, or CAN protocols. Peripheral controller chips handle protocol conversion.
- Electrical Isolation: Industrial embedded systems connect to high-voltage machinery. Optocouplers and isolation transformers protect the MCU from high voltages.
- Impedance Matching: Ensures proper electrical loading so signal integrity is maintained across interconnections.
- Fan-out/Fan-in Management: A single MCU pin may not drive many loads directly; buffer ICs (74HC244) increase drive capability.

Types of Interfacing:

- Memory Interfacing: Connecting RAM, ROM, and Flash memory chips to the processor address and data buses.
- Peripheral Interfacing: Connecting I/O devices (keyboard, display, ADC, DAC, sensors) to the microprocessor.
- Serial Communication Interfacing: UART, SPI, I2C, CAN, USB — protocols for serial data exchange between ICs or systems.
- Interrupt Interfacing: Hardware circuits that signal the CPU when a peripheral needs attention.
- DMA Interfacing: Hardware (DMA controller) enabling direct high-speed data transfer between peripheral and memory without CPU involvement.

- Analog Interfacing: ADC (Analog-to-Digital Converter) and DAC (Digital-to-Analog Converter) connecting analog sensors/actuators to digital processor.

Q2. What are the differences between memory-mapped I/O and isolated (port-mapped) I/O? Give examples of each.	**2079 Ashwin (Back) | 2071 Bhadra (Back) [3+5]**

ANSWER

Aspect	Memory-Mapped I/O	Isolated (Port-Mapped) I/O
Address Space	Peripherals share the main memory address space	Separate I/O address space (independent of memory)
Instructions	Normal load/store (LDR, STR, MOV) instructions access I/O registers	Special IN/OUT instructions required (x86: IN AL,port / OUT port,AL)
Address Decoding	Same decoder for memory and I/O	Separate I/O address decoder; IO/M# pin distinguishes access type
Memory Impact	Reduces available memory address space	Does not consume memory address space
Flexibility	Any instruction (arithmetic, logic) can operate on I/O registers	Only IN/OUT instructions; I/O registers less flexible
Architecture	Used in RISC architectures (ARM, MIPS)	Used in CISC (x86, 8051 XRAM)
Example	ARM Cortex-M: GPIOA_ODR at 0x40020014 accessed by STR instruction	x86: IN AL, 0x60 reads keyboard port; OUT 0x61, AL writes port

```
// ARM Cortex-M — Memory-Mapped I/O example
#define GPIOA_ODR  (*(volatile uint32_t *)0x40020014)
GPIOA_ODR |= (1 << 5);  // Toggle PA5 using normal store instruction

; x86 — Isolated I/O example
IN  AL, 0x60   ; Read keyboard scan code from I/O port 0x60
OUT 0x61, AL   ; Write to I/O port 0x61 (speaker control)
```

Q3. Explain interrupt and explain its various types in detail.	**2081 Ashwin (Back) | 2075 Bhadra (Back) | 2073 Magh (Regular) [3+5]**

ANSWER

Interrupt:

An interrupt is an asynchronous signal (from hardware or software) that causes the CPU to suspend its current execution, save its context (PC, status register, registers) on the stack, and transfer control to a special routine called an Interrupt Service Routine (ISR). After the ISR completes, the CPU restores its saved context and resumes the interrupted program.

Types of Interrupts:

1. Hardware Interrupt (External Interrupt): Generated by external hardware devices via the IRQ (Interrupt Request) line. Examples: UART data-received interrupt, Timer overflow interrupt, ADC conversion-complete interrupt, GPIO edge interrupt.

2. Software Interrupt (Trap): Generated by executing a special instruction (INT n in x86, SVC/SWI in ARM). Used for system calls to OS services (open file, allocate memory). Also used for debugging (breakpoint = INT 3 in x86).

3. Maskable Interrupt: Can be enabled or disabled (masked) by the programmer using the Interrupt Enable (IE) flag in the status register. Used for non-critical events. Most hardware interrupts are maskable.

4. Non-Maskable Interrupt (NMI): Cannot be disabled by software. Used for critical, high-priority events that must always be serviced: power failure detection, memory parity error, hardware watchdog timeout, hardware fault.

5. Vectored Interrupt: Each interrupting device supplies its own interrupt vector number (an index into the interrupt vector table). The CPU uses this to directly jump to the correct ISR without polling. Faster response time.

6. Non-Vectored Interrupt: All interrupting devices share one ISR address. The ISR must poll each device to identify the source of the interrupt. Simpler hardware, slower response.

7. Level-Triggered Interrupt: The IRQ line is held at a specific level (HIGH or LOW) until the interrupt is serviced. Risk of repeated triggering if not cleared promptly.

8. Edge-Triggered Interrupt: The interrupt is triggered by a rising or falling edge transition of the IRQ signal. Only one event per edge — avoids repeated triggering. Used in most modern MCUs.

9. Internal Interrupt (Exception): Generated by CPU itself in response to an error condition: divide-by-zero, illegal instruction, memory access violation, stack overflow. ARM calls these 'exceptions'.

Q4. Explain interrupt-driven I/O using vectored interrupt with the summary of flow of actions. Compare interrupt-driven I/O with polling.	**2081 Chaitra (Regular) | 2079 Chaitra (Back) | 2078 Baishakh (Back) [4+4]**

ANSWER

Interrupt-Driven I/O using Vectored Interrupt:

```
                    SYSTEM BUS
                 Address/Data/Control
┌──────────┐                          ┌──────────────┐
│   CPU    │◄────────────────────────►│  Peripheral│
│          │                          │(UART/ADC..)│
│  INTACK  │◄── IRQ ──────────────────│            │
│          │─── INTA ────────────────►│            │
└──────────┘                          └──────────────┘
     │
Interrupt Vector Table
[Vector 0] → ISR_0 address
[Vector 1] → ISR_1 address
[Vector n] → ISR_n address  ← peripheral supplies n
```

Flow of Actions (Vectored Interrupt Sequence):

1. Peripheral completes operation (e.g., UART receives a byte) and asserts IRQ line HIGH.
2. CPU finishes executing current instruction (cannot interrupt mid-instruction).
3. CPU checks interrupt enable flag (IE); if enabled, acknowledges by asserting INTA (Interrupt Acknowledge) signal.
4. The interrupting device places its interrupt vector number (e.g., 0x28) on the data bus.
5. CPU automatically saves PC (return address) and Status Register (PSW) onto the stack.
6. CPU computes ISR address = base_of_vector_table + vector_number × 4; fetches ISR address.
7. CPU loads PC with ISR address → begins executing ISR.
8. ISR reads data from peripheral (e.g., UART data register), clears interrupt flag, processes data.
9. ISR executes RETI (Return from Interrupt) instruction.
10. CPU restores PC and PSW from stack → resumes interrupted main program at exact point.

Comparison — Interrupt-Driven I/O vs Polling:

Property	Polling	Interrupt-Driven I/O
Mechanism	CPU repeatedly reads peripheral status register in a loop	Peripheral signals CPU via IRQ when ready
CPU utilization	Wasted — CPU busy even when peripheral not ready	Efficient — CPU runs main code; ISR only on event
Response latency	Variable (depends on polling loop frequency)	Low, deterministic (interrupt latency)

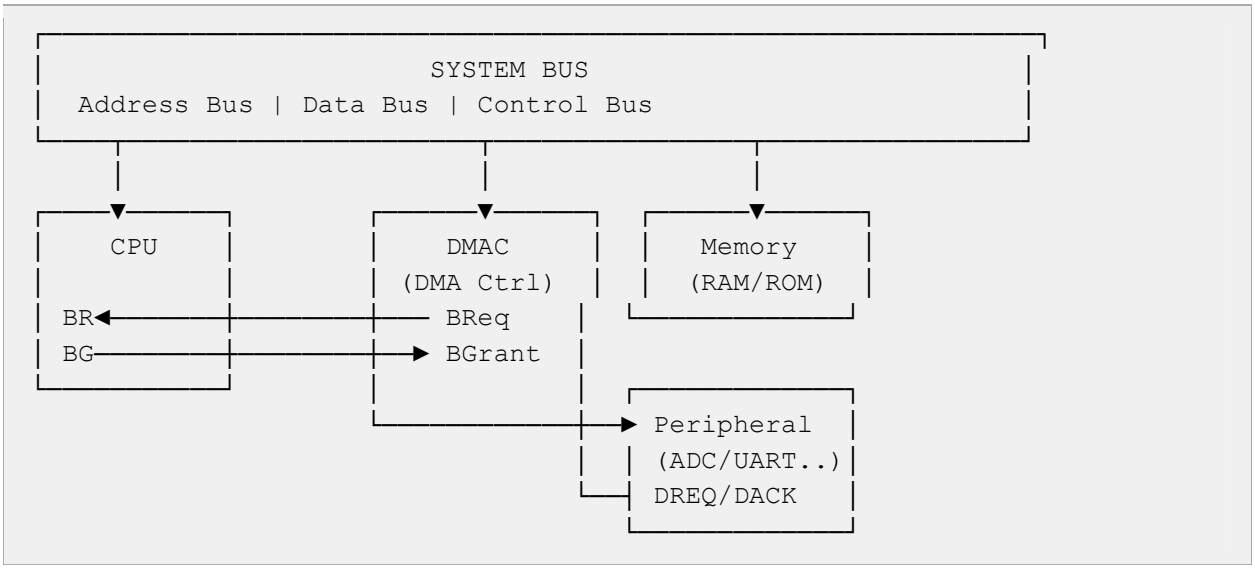
Power consumption	High (CPU always active)	Lower (CPU can sleep between events)
Complexity	Simple to implement	More complex (ISR, vector table, shared data)
Suitable for	High-frequency events, few peripherals	Infrequent/async events, many peripherals
Example	Checking button state in main loop	UART receive, ADC complete, timer tick

Q5. Explain microprocessor interfacing using DMA with necessary block diagram. Explain DMA transfer modes.	**2080 Ashwin (Back) | 2081 Chaitra (Regular) | 2079 Jestha (Back) [4+4]**

ANSWER

DMA (Direct Memory Access):

DMA is a technique that allows a DMA controller (DMAC) to transfer data directly between a peripheral and memory — or between two memory regions — without CPU involvement in each transfer. The CPU initializes the DMAC and is freed during the actual data movement.



DMA Operation Sequence:

1. CPU programs DMAC: source address register (SAR), destination address register (DAR), count register (CR), control register (read/write direction, mode).
2. Peripheral asserts DREQ (DMA Request) to DMAC when it has data ready (or needs data).
3. DMAC asserts BR (Bus Request) to CPU.

4. CPU completes current memory cycle, then asserts BG (Bus Grant) and tri-states (releases) the buses.
5. DMAC takes control of Address, Data, and Control buses.
6. DMAC reads from source (peripheral/memory) and writes to destination (memory/peripheral). Increments SAR/DAR, decrements CR.
7. When CR = 0 (all bytes transferred), DMAC releases bus (de-asserts BR). Optionally asserts interrupt to CPU to signal completion.

DMA Transfer Modes:

1. Burst Mode (Block Transfer Mode): DMAC holds the bus for the entire block transfer — CPU has no bus access until complete. Fastest throughput but CPU is frozen. Used for high-speed bulk transfers (disk to RAM).

2. Cycle-Steal Mode (Single Transfer Mode): DMAC steals one bus cycle at a time, then releases the bus back to CPU. CPU and DMAC alternate. CPU is slowed but not completely blocked. Used when CPU must remain responsive.

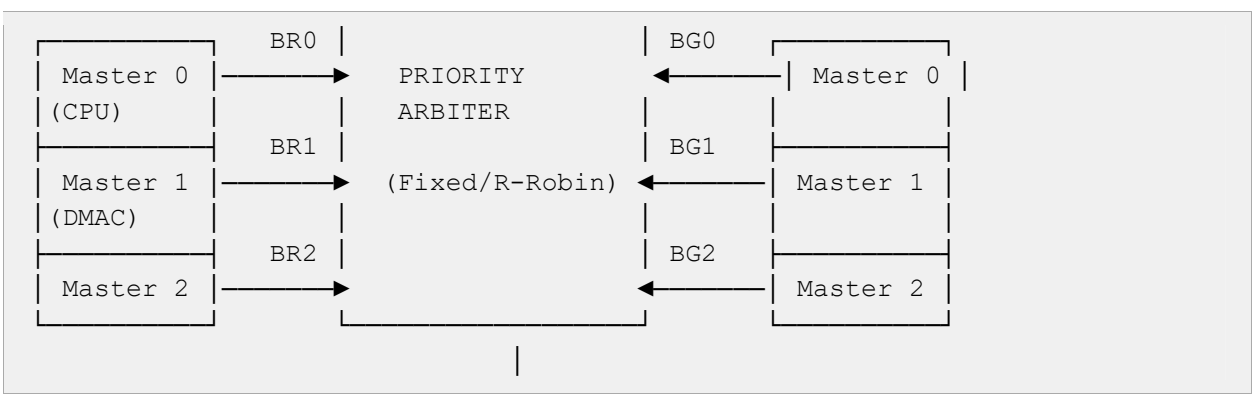
3. Transparent Mode (Hidden Mode): DMAC transfers only when CPU is not using the bus (e.g., during internal ALU operations that don't need the bus). CPU performance unaffected but transfer rate depends on CPU activity.

Q6. Explain Priority Arbitration with a block diagram. Describe fixed priority and round-robin arbitration.	**2080 Chaitra (Regular) | 2079 Ashwin (Back) | 2077 Chaitra (Back) [1+2+5]**

ANSWER

Bus Arbitration:

When multiple bus masters (CPU, DMAC, multiple processors) need the system bus simultaneously, a bus arbitration mechanism decides which master gets access. The goal is to resolve conflicts fairly and efficiently.



```
                    BB (Bus Busy) ← asserted by granted master
```

Fixed Priority Arbitration:

Each master is assigned a permanent priority level (Master 0 = highest, Master n = lowest). The arbiter always grants the bus to the highest-priority requesting master.

1. Each master asserts its BR (Bus Request) line when it needs the bus.
2. Arbiter scans all BR lines and identifies the highest-priority active request.
3. Arbiter asserts BG (Bus Grant) to that master only.
4. Granted master asserts BB (Bus Busy) and takes control of the bus.
5. Other masters wait. When BB is de-asserted, arbitration repeats.

Advantage: Simple, fast, deterministic. Disadvantage: Low-priority masters may starve (never get access) if high-priority masters continuously request the bus.

Round-Robin Arbitration:

Priority rotates after each grant. A rotating pointer advances to the next master after each grant, giving every master an equal chance.

1. After granting to Master i, the priority pointer advances to Master (i+1) mod N.
2. The next grant goes to the highest-priority requester starting from the pointer position.

Advantage: Fair — no starvation. Each master gets periodic access. Disadvantage: Critical masters (like interrupt controllers) may be delayed.

Q7. Discuss in brief about Daisy Chain Arbitration. Draw its block diagram and explain the steps.	**2082 Kartik (Back) | 2080 Ashwin (Back) | 2078 Baishakh (Back) | 2076 Baisakh (Back) [4+4]**

ANSWER

Daisy Chain Arbitration:

Daisy chain arbitration is a bus arbitration scheme where multiple bus masters are connected in a serial chain. A single Bus Grant (BG) signal propagates from one device to the next — the first device in the chain that needs the bus captures the grant.

```
        BG_in          BG_out      BG_in          BG_out     BG_in
CPU ──────────▶ Device 1 ──────────────▶ Device 2 ──────────────▶ Device 3
 │            │     (Highest        │               │            (Lowest
 │            │      Priority)      │               │            Priority)
 │            │                     │               │
 │◀── BR ─────┘                     │               │
 │◀── BR ──────────────────────────┘               │
```

```
    |◄── BR ──────────────────────────────────────┘
    |
    |─── BG ──────────────────────────────────────► (chain starts)
    |
    |◄── BB (Bus Busy) ── any device asserting bus busy
```

Operation Steps:

1. Any device needing the bus asserts its BR (Bus Request) line to the CPU.
2. CPU asserts BG (Bus Grant) signal — this propagates into the chain starting at Device 1.
3. Device 1 checks if it needs the bus. If YES: Device 1 captures the grant (stops propagation), asserts BB (Bus Busy), and uses the bus. If NO: Device 1 passes BG_out to Device 2.
4. This propagates until the first device that needs the bus captures it.
5. When done, that device de-asserts BB, freeing the bus for next arbitration cycle.

Properties:

- Priority: Devices closer to CPU have higher priority (Device 1 > Device 2 > Device 3...).
- Hardware simplicity: Only one BG signal line regardless of number of devices — scales easily.
- Disadvantage 1: If a device fails (or is removed), all lower-priority devices are blocked.
- Disadvantage 2: Priority is fixed by physical position — cannot be changed dynamically.
- Disadvantage 3: Grant propagation takes time proportional to chain length — higher latency for low-priority devices in long chains.

Q8. Describe the significance of I2C serial communication protocol. Explain its frame format and timing.	**2082 Kartik (Back) | 2070 Bhadra (Back) | 2076 Chaitra (Back) [4+4]**

ANSWER

I2C (Inter-Integrated Circuit) Protocol:

I2C is a two-wire synchronous serial communication protocol developed by Philips (NXP). It enables multiple ICs to communicate on a shared bus using only two lines.

```
┌──────────────┐        SDA (Serial Data)        ┌──────────┐   ┌──────────┐
│   MASTER     │◄──────────────────────────────►│ SLAVE 1  │   │ SLAVE 2  │
│ (MCU/CPU)    │◄──────────────────────────────►│(EEPROM)  │   │(Sensor)  │
│              │        SCL (Serial Clock)       └──────────┘   └──────────┘
```

```
└──────────────┘
         │                 │
         Vcc (3.3V/5V) ── Pull-up resistors (4.7kΩ typical)
         │                 │
        SDA               SCL   ← Open-drain lines, need pull-ups
```

Key Features:

- Two Lines Only: SDA (Serial Data Line) and SCL (Serial Clock Line). Both are open-drain (active LOW, released HIGH by pull-up resistors).
- Master-Slave: One or more masters generate SCL; multiple slaves identified by unique 7-bit (or 10-bit) addresses.
- Multi-master with Collision Detection: If two masters drive SDA simultaneously, the loser detects the collision and backs off.
- Speeds: Standard Mode = 100 kbps; Fast Mode = 400 kbps; Fast-Plus = 1 Mbps; High-Speed = 3.4 Mbps.

I2C Frame Format:

```
START  [7-bit Address][R/W] ACK  [8-bit Data] ACK  [8-bit Data] ACK  STOP
  S   A6 A5 A4 A3 A2 A1 A0  R/W  A  D7..D0   A   D7..D0   A    P

START Condition:  SDA falls while SCL is HIGH
STOP  Condition:  SDA rises while SCL is HIGH
ACK:              Receiver pulls SDA LOW during 9th clock =
acknowledgement
NACK:             SDA left HIGH during 9th clock = no acknowledgement

Write sequence: S + [Addr + W] + ACK + [Data byte] + ACK + ... + P
Read  sequence: S + [Addr + R] + ACK + [Data byte] + ACK + ... + NACK + P
```

Applications (Significance):

- EEPROMs (AT24Cxx), real-time clocks (DS1307), temperature sensors (LM75), accelerometers (MPU-6050), display controllers (SSD1306 OLED).
- Ideal for low-speed, short-distance, multi-device communication on a single PCB.

Q9. Explain SPI (Serial Peripheral Interface) communication protocol. Compare SPI with I2C.	**Model Question | 2078 Poush (Back) | 2074 Jestha (Back) [4+4]**

ANSWER

SPI (Serial Peripheral Interface):

SPI is a four-wire synchronous full-duplex serial communication protocol. It uses a master-slave topology where one master can communicate with multiple slaves using individual Chip Select (CS) lines.

```
┌──────────────┐  SCLK (Clock)      ┌──────────┐
│              │──────────────────→│          │
│   MASTER     │  MOSI (Master Out)│ SLAVE 1  │
│ (MCU/FPGA)   │──────────────────→│          │
│              │  MISO (Master In) │          │
│              │←──────────────────│          │
│              │  CS1 (active LOW) │          │
│              │──────────────────→│          │
└──────────────┘                   └──────────┘
        │  CS2 ──────────────────→ SLAVE 2
        │  CS3 ──────────────────→ SLAVE 3
```

SPI Lines:

- SCLK (Serial Clock): Master generates clock; slaves synchronize to it.
- MOSI (Master Out Slave In): Data from master to slave.
- MISO (Master In Slave Out): Data from slave to master (full duplex).
- CS/SS (Chip Select / Slave Select): Active LOW signal from master selects a specific slave. One CS line per slave.

SPI Data Transfer:

Master asserts CS LOW for the target slave, then generates SCLK pulses. Data is shifted out on MOSI and simultaneously shifted in on MISO — 8, 16, or 32 bits per transfer. No addressing — the CS line selects the device.

SPI vs I2C Comparison:

Property	SPI	I2C
Wires	4 (SCLK, MOSI, MISO, CS per slave)	2 (SDA, SCL) regardless of slaves
Duplex	Full duplex (simultaneous TX and RX)	Half duplex (one direction at a time)
Speed	Up to 100+ MHz	Up to 3.4 Mbps (High-Speed mode)
Addressing	No addressing — CS line selects slave	7-bit or 10-bit address in frame
Multi-master	Not standard	Supported with collision detection
Hardware	Simpler slave hardware	Open-drain + pull-ups required
Distance	Short PCB traces (cm–m)	Short PCB traces (cm–m)
Typical use	ADC, DAC, Flash, SD card, displays	EEPROM, RTC, sensors, low-speed ICs

Q10. Explain UART (Universal Asynchronous Receiver/Transmitter) communication. Describe its frame format.	**Model Question | 2077 Ashwin (Back) | 2073 Baisakh (Back) [4+4]**

ANSWER

UART (Universal Asynchronous Receiver/Transmitter):

UART is an asynchronous serial communication protocol. Unlike SPI and I2C, it has no shared clock — sender and receiver must be configured to the same baud rate (bits per second) beforehand. It uses two wires: TX (Transmit) and RX (Receive).

```
Device A                                    Device B
┌──────────────┐    TX ──────────────►    ┌──────────────┐
│  UART        │    RX ◄──────────────    │  UART        │
│  Transmitter │                          │  Receiver    │
└──────────────┘                          └──────────────┘

   (No shared clock — both use same baud rate setting)
```

UART Frame Format:

```
IDLE   START   D0   D1   D2   D3   D4   D5   D6   D7   PARITY   STOP(s)
 HIGH    0    LSB                                MSB   (opt)   HIGH


─────┐                                                        ┌────
     │  0 |D0|D1|D2|D3|D4|D5|D6|D7| P |  1  |  1  |
     └──────────────────────────────────────────────────────┘
     START                                                STOP

Baud rate = bits/second. Common rates: 9600, 115200, 1M, 3M bps
Baud rate tolerance: receiver must be within ±3% of transmitter
```

Frame Components:

- Idle state: TX line held HIGH when no transmission.
- Start bit: Single LOW bit signals beginning of frame.
- Data bits: 5–9 bits, LSB first. Typically 8 bits.
- Parity bit (optional): Even or odd parity for basic error detection.
- Stop bits: 1 or 2 HIGH bits — marks end of frame; gives receiver time to process.

UART vs RS-232 vs RS-485:

- UART: Logic-level (3.3V/5V) serial signal — for IC-to-IC communication.
- RS-232: Voltage-shifted UART (±12V levels) for longer cable distances (DB9 connector, PC serial port).
- RS-485: Differential signaling, supports multi-drop networks (up to 32 devices), 1200m cable. Used in industrial (Modbus).

Q11. Explain communication basics in embedded systems. Differentiate between serial and parallel communication, and between synchronous and asynchronous communication.	Model Question | 2076 Jestha (Back) | 2074 Chaitra (Back) [4+4]

ANSWER

Communication Basics:

Communication in embedded systems refers to the exchange of data between a processor, memory, and peripherals, or between two separate embedded systems. Key concepts:

- Simplex: Data flows in one direction only (e.g., TV remote → TV).
- Half Duplex: Data flows in both directions, but only one direction at a time (e.g., I2C, RS-485 Modbus).
- Full Duplex: Data flows in both directions simultaneously (e.g., SPI, UART with TX+RX, Ethernet).
- Bit Rate (bps): Number of bits transmitted per second.
- Baud Rate: Number of signal changes per second. Equal to bit rate for binary (2-level) signaling.

Serial vs Parallel Communication:

Property	Serial Communication	Parallel Communication
Data bits	1 bit at a time on 1 wire	Multiple bits simultaneously on multiple wires
Wires	Few (1–4)	Many (8, 16, or 32 data lines + control)
Speed	Lower per cycle but high frequency possible	Higher per cycle but limited by skew at high freq
Cost	Low (fewer pins/wires)	High (many pins, wide bus, complex PCB)
Distance	Long distance possible	Short distances (PCB-level only at high speed)
Noise immunity	Better (fewer lines to pick up noise)	Worse (many parallel lines, crosstalk)
Examples	UART, SPI, I2C, USB, CAN	CPU data bus, memory bus, GPIO port

Synchronous vs Asynchronous Communication:

Property	Synchronous	Asynchronous
Clock	Shared clock line between sender and receiver	No shared clock — each device uses its own
Timing	Data valid on clock edges (rising/falling)	Start/stop bits frame each data byte
Overhead	No framing overhead — efficient for bulk data	Start + stop bits add ~20% overhead per byte
Speed	Higher (clock can be as fast as hardware allows)	Lower (limited by baud rate tolerance)
Complexity	Requires clock distribution (skew issues at high speed)	Simpler — no clock line needed

Examples	SPI, I2C, CAN, USB, Ethernet	UART/RS-232, RS-485

Q12. Explain multilevel bus architecture with a diagram. Why is it used in embedded systems?	**Model Question | 2078 Baishakh (Back) | 2075 Poush (Back) [4+4]**

ANSWER

Multilevel Bus Architecture:

A multilevel (hierarchical) bus architecture organizes the system bus into multiple levels with different speeds and bandwidths. Fast components connect to fast buses; slow peripherals connect to slow buses. Bridges connect bus levels.

```
┌──────────────────────────────────────────────────────────────────┐
│                    LEVEL 1: PROCESSOR BUS (Fastest)              │
│  CPU ◄──────────────────────────────────────► L1/L2 Cache       │
│              ~GHz, wide data bus (64-256 bit)                    │
└──────────────────────────────────────────────────────────────────┘
                           │  Memory Controller / Host Bridge
┌──────────────────────────▼───────────────────────────────────────┐
│                    LEVEL 2: SYSTEM BUS (Fast)                    │
│  DRAM Controller ◄──────────────────────► PCIe / AHB            │
│              ~100-400 MHz, 32-64 bit                             │
└──────────────────────────────────────────────────────────────────┘
                           │  Peripheral Bridge (AHB-to-APB)
┌──────────────────────────▼───────────────────────────────────────┐
│                    LEVEL 3: PERIPHERAL BUS (Slow)                │
│  UART ◄──► SPI ◄──► I2C ◄──► GPIO ◄──► Timers ◄──► ADC          │
│              ~1-50 MHz, 32-bit APB, low power                    │
└──────────────────────────────────────────────────────────────────┘
```

Why Multilevel Bus Architecture is Used:

- Performance isolation: Fast devices (CPU, cache, DRAM) are not slowed down by slow peripherals (UART, I2C). They communicate on separate high-speed buses.
- Power efficiency: Peripheral bus runs at lower frequency → lower dynamic power. Peripheral bus can be clock-gated when unused.
- Cost: Wide, high-speed buses are expensive in silicon area. Using narrow, slow buses for low-speed peripherals reduces chip area.
- Scalability: New peripherals are added to the peripheral bus without affecting the processor bus performance.
- Example in ARM Cortex-M: AHB (Advanced High-performance Bus) for CPU, memory, DMA; APB (Advanced Peripheral Bus) for UART, SPI, I2C, GPIO via AHB-to-APB bridge.

Q13. Explain CAN (Controller Area Network) bus protocol used in embedded systems. Where is it applied?	Model Question | 2077 Baisakh (Back) | 2076 Poush (Back) [4+4]

ANSWER

CAN (Controller Area Network) Bus:

CAN is a robust, multi-master serial communication protocol designed for real-time control applications in noisy environments. Originally developed by Bosch in 1983 for automotive ECU networking.

```
┌──────────┐                                          ┌──────────┐
│  Node 1  │  CANH ──────────────────────────────  CANH │  Node 3  │
│ (Engine  │  CANL ──────────────────────────────  CANL │ (ABS     │
│  ECU)    │   │   ┌──────────┐                     │   │  ECU)    │
└──────────┘   │   │  Node 2  │                     │   └──────────┘
               │   │(Dash ECU)│                     │
120Ω           │   └──────────┘                     │          120Ω
terminator  CAN Bus (Differential pair)        terminator
```

Key Features:

- Two-wire differential bus: CANH and CANL. Differential signaling provides excellent noise immunity.
- Multi-master: Any node can transmit when the bus is idle. Collision resolved by arbitration (lower CAN ID = higher priority).
- Message-based (not address-based): Each message has an 11-bit or 29-bit identifier. All nodes receive all messages; each node filters by ID.
- Non-destructive bitwise arbitration: Higher priority message wins without corruption of lower priority messages.
- Built-in error detection: CRC checksum, bit stuffing, ACK, frame check — very high reliability.
- Speeds: CAN 2.0A/B: up to 1 Mbps. CAN FD: up to 8 Mbps (flexible data rate).

CAN Frame Format (Standard 2.0A):

```
SOF | Arbitration (11-bit ID) | Control | Data (0-8 bytes) | CRC | ACK |
EOF
 1b |         11b + RTR       |    6b    |    0-64 bits     | 15b | 2b  |
7b
```

Applications:

- Automotive: Engine ECU, ABS, airbag, dashboard, transmission control — all interconnected on CAN.
- Industrial automation: Factory floor controllers, Modbus-over-CAN, CANopen protocol.
- Medical devices: Patient monitoring systems requiring reliable real-time communication.
- Aerospace: CAN is used in small aircraft avionics (ARINC 825 uses CAN).

Q14. Give an overview of USB (Universal Serial Bus) in the context of embedded systems. Explain USB device classes and speeds.	Model Question | 2075 Chaitra (Back) [4+4]

ANSWER

USB (Universal Serial Bus):

USB is a standard interface for connecting peripheral devices to a host (PC, embedded SoC). It provides power delivery, data transfer, and hot-plug capability over a single cable.

```
USB Host (PC / SoC)                          USB Device
┌──────────────────┐   VBUS (+5V)          ┌──────────────┐
│  USB Host        │──────────────────────►│ USB Function │
│  Controller      │   D+ (data+)          │ Controller   │
│                  │◄──────────────────────►│              │
│                  │   D- (data-)          │              │
│                  │◄──────────────────────►│              │
│                  │   GND                 │              │
└──────────────────┘──────────────────────►└──────────────┘
```

USB Speeds:

Version	Speed	Use Case
USB 1.0 Low Speed	1.5 Mbps	Mouse, keyboard
USB 1.0 Full Speed	12 Mbps	Audio devices, serial adapters
USB 2.0 High Speed	480 Mbps	Cameras, storage, most embedded USB
USB 3.0 SuperSpeed	5 Gbps	Hard drives, video capture
USB 3.1 SuperSpeed+	10 Gbps	High-speed storage, docking stations

USB Device Classes (common in embedded):

- CDC (Communication Device Class): Implements virtual COM port — MCU appears as a serial port to PC. Most common in microcontroller USB implementations (STM32 USB-CDC).
- HID (Human Interface Device): Keyboard, mouse, joystick, custom USB HID devices. No custom driver needed.
- MSC (Mass Storage Class): MCU exposes Flash/SD card as USB drive.
- DFU (Device Firmware Upgrade): USB-based firmware update protocol — used to update MCU firmware without hardware programmer.
- Audio Class: Microphone, speaker, USB sound card.

Summary: Topics Covered in Chapter 5

S.N.	Topic	Exam(s) Asked	Marks
1	Definition, Needs, and Types of Interfacing	2080 Chaitra, 2075 Jestha	1+3+4
2	Memory-Mapped I/O vs Isolated I/O	2079 Ashwin, 2071 Bhadra, 2078 Poush	3+5
3	Types of Interrupts (9 types)	2081 Ashwin, 2075 Bhadra, 2073 Magh	3+5
4	Interrupt-Driven I/O + Vectored Interrupt Flow	2081 Chaitra, 2079 Chaitra, 2078 Baishakh	4+4
5	Polling vs Interrupt-Driven I/O (comparison)	2081 Chaitra, 2079 Chaitra, Multiple	4
6	DMA — Operation, Block Diagram, 3 Modes	2080 Ashwin, 2081 Chaitra, 2079 Jestha	4+4
7	Priority Arbitration (Fixed + Round-Robin)	2080 Chaitra, 2079 Ashwin, 2077 Chaitra	1+2+5
8	Daisy Chain Arbitration (block diagram + steps)	2082 Kartik, 2080 Ashwin, 2078 Baishakh, 2076	4+4
9	I2C Protocol (frame format, timing, applications)	2082 Kartik, 2070 Bhadra, 2076 Chaitra	4+4
10	SPI Protocol + SPI vs I2C	2078 Poush, 2074 Jestha, Model Q	4+4
11	UART Protocol (frame format, RS-232, RS-485)	2077 Ashwin, 2073 Baisakh, Model Q	4+4
12	Serial vs Parallel / Synchronous vs Asynchronous	2076 Jestha, 2074 Chaitra, Model Q	4+4
13	Multilevel Bus Architecture (AHB/APB/PCIe)	2078 Baishakh, 2075 Poush, Model Q	4+4
14	CAN Bus Protocol + Applications	2077 Baisakh, 2076 Poush, Model Q	4+4
15	USB — Speeds, Device Classes	2075 Chaitra, Model Q	4+4